#A Initialize HMAC-SHA256 with the authentication key.
#B Compute an initial tag over the macaroon identifier.
#C Compute a new tag for each caveat using the old tag as the key.
#D Compare the tags with a constant-time equality function.

After the HMAC tag has been verified, the API then needs to check that the caveats are satisfied. There's no standard set of caveats that APIs support, so like OAuth2 scopes it's up to the API designer to decide what to support. There are two broad categories of caveats supported by macaroon libraries:

- *First-party caveats* are restrictions that can be easily verified by the API at the point of use, such as restricting the times of day at which the token can be used. First-party caveats are discussed in more detail in section 9.3.1.
- *Third-party caveats* are restrictions which require the client to obtain a proof from a 3rd-party service, such as proof that the user is an employee of a particular company or that they are over 18. Third-party caveats are discussed in section 9.3.2.

A significant advantage of macaroons over other token forms is that they allow the client to attach *contextual caveats* just before the macaroon is used. For example, a client that is about to send a macaroon to an API over an untrustworthy communication channel can attach a first-party caveat limiting it to only be valid for HTTP PUT requests for the next 5 seconds. That way if the macaroon is stolen then the damage is limited because the attacker can only use the token in very restricted circumstances. Because